

HIGHER SCHOOL OF ECONOMICS NATIONAL RESEARCH UNIVERSITY

Faculty of Computer Science

Educational programme «Data Science and Business Analytics»

Project proposal

“Development of a Web Application for Portfolio Management”

Fulfilled by:

Bratukhin Nikita Alexeyevich DSBA232, 3rd year

Rodikov Maksim Dmitrievich DSBA232, 3rd year

Moscow 2026

1. Introduction and general vision of the project	2
1.1 Problem	2
1.2 The emergence of an idea	2
1.3 Overall vision	3
2. Analysis of available solutions	3
2.1 How the dataset was assembled	3
2.2 Our assumption	3
2.3 Dataset description	4
2.4 Analysis	6
3. How we implement the project	6
3.1 Frontend	6
3.2 Backend	6
3.3 General structure	6
4. Selected analytical framework	7
5. Project success metrics	9
5.1. Popularity and distribution metrics	9
5.2 Community activity metrics	9
5.3 Final success criteria	9
6. Project objectives and expected results	10
7. References	11

1. Introduction and general vision of the project

1.1 Problem

Our team is developing a portfolio management app. This idea is nothing new and has been implemented numerous times by numerous companies. However, as we've discovered, every implementation suffers from one of several significant flaws, making them useless for some users (especially in the Russian market).

We discovered this by analyzing the dataset of alternative options we compiled, which will be shown below.

1.2 The emergence of an idea

We decided to create the project primarily because we ourselves had created the problem. I personally try to consider all the possible funds at my disposal, but I couldn't find a solution that would meet all the logical objectives and wouldn't cost too much.

The only way to solve this problem is to manage multiple Google spreadsheets, which is extremely inconvenient and has a number of its own drawbacks, although it is superior to other options.

1.3 Overall vision

In general, this software should allow you to add and edit transactions, like in any financial management app. Transactions should be merged into positions, which in turn are part of the user's overall portfolio. Based on all of the above, price and market conditions should be tracked over time for subsequent analysis using charts.

The solution must meet several key criteria, which we believe are the most important. We will discuss them in the following subsections of this section.

Cross-platform

Cross-platform access is essential for any device. Mobile-focused solutions are inconvenient for tracking and analyzing data when you have a laptop or PC at hand, but ignoring phones is also unavoidable these days.

Self-hosted implementation

Self-hosted implementation is required to support openness (we do not charge users) and data privacy.

This eliminates the possibility of mass leaks of crucial information about users' accounts and the amount of money in them; everyone is responsible for their own safety.

Open-source implementation

The openness of the code logically follows from the fact that it should be located on users' own machines, but it is also worth noting one important advantage that is due to openness: modularity.

Modularity here means the ability to customize the code and create an environment suited to specific groups of people. In the future, it would be desirable to create a custom add-on system (Obsidian's experience is an example). As AI develops, this factor will only become more important, since almost anyone (still a programmer) can customize open-source solutions and create add-ons easily and in a short amount of time.

Support for all available assets

This is necessary to solve the main problem: monitoring all assets in one place. This includes cryptocurrencies, stocks, bonds, futures, and other traditional financial instruments.

It is also worth noting the required ability to create custom assets, such as a car or apartment, to track their changes over time.

Evidence-based financial analytics

It's worth noting that this task is being developed primarily by the finance team. It's important to us as a killer feature of our solution, as it's not used anywhere in non-professional platforms, which often rely on useless TA and the like. We'll discuss the planned methodologies in more detail in the section on evidence-based financial analytics research.

2. Analysis of available solutions

In this section, we'll examine a small dataset we've compiled of solutions to the portfolio analytics problem. It can be accessed via the link provided at the end of the document.

2.1 How the dataset was assembled

Since there is no aggregated data suitable for analytics in our case, we collected our own dataset through LLM Research [\[1\]](#).

LLM was asked to search for all possible portfolio analytics solutions, indicating where the service was found and links to its main page. All of this was then compiled into a CSV file, with some custom solutions added. The file containing service names and links was also supplemented with empty user fields, which LLM filled in based on available information on the websites and articles where they were found.

In the end, we went through the information we received and were partially convinced of its veracity.

2.2 Our assumption

The dataset was almost entirely compiled by LLM; this isn't definitive information, but it can demonstrate the general state of the portfolio analytics market and the absence of a solution similar to ours (we didn't remove any data from what LLM found).

2.3 Dataset description

The dataset consists of 86 identified services related to portfolio analytics. Below are the features described for each solution.

Name

Name of the service.

Category

General categorization of the presented solutions. Each service is assigned to one of four categories:

1. Universal_tracker - Specialized services for tracking investment portfolios.
2. Crypto_cefi - Cryptocurrency trackers operating through centralized exchanges.
3. Defi_web3 - DeFi trackers/exchanges.

4. Institutional - professional terminals for funds, banks, etc.
5. Net_worth_aggregator - aggregators of all assets, closest to our idea.
6. Social_trading - platforms with the ability to interact with people.
7. Spreadsheet - table-based solutions.

Description

A general description of the LLM service for verifying key parameters for various errors.

Platforms

Platforms on which the solution is available. There are six platforms in total: web, mobile, desktop, API, Mac, and iOS.

Price_min_usd_mo and price_max_usd_mo

Subscription prices are in dollars. For enterprise solutions, they are inf; for free ones, they are 0. These are the most problematic fields, as information is often unavailable without registration and can also be charged as transaction fees by brokers, which is much more difficult to estimate.

Prices_detail и comission

The parameters described by LLM are used to verify price data. Brokers can be found based on their commissions.

Is_self_hosted, assets_[stock, stock_ru, crypto, custom]

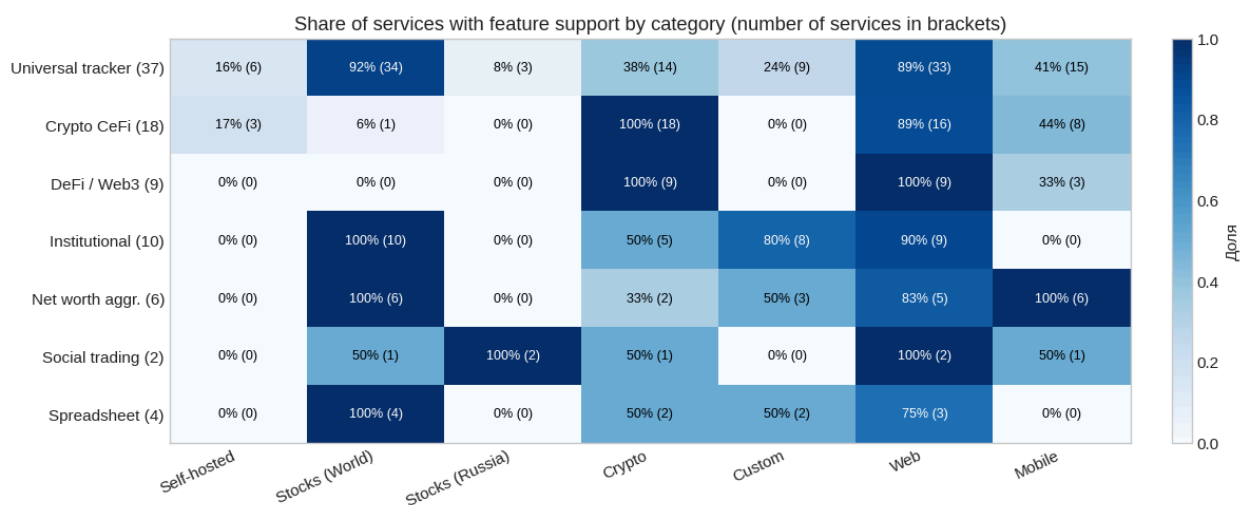
Yes/no fields indicating whether the service can be deployed on your own server and what assets it supports.

Url and url_found

Link to the main page of the solution and the site where it was discovered by LLM during the first stage of dataset collection.

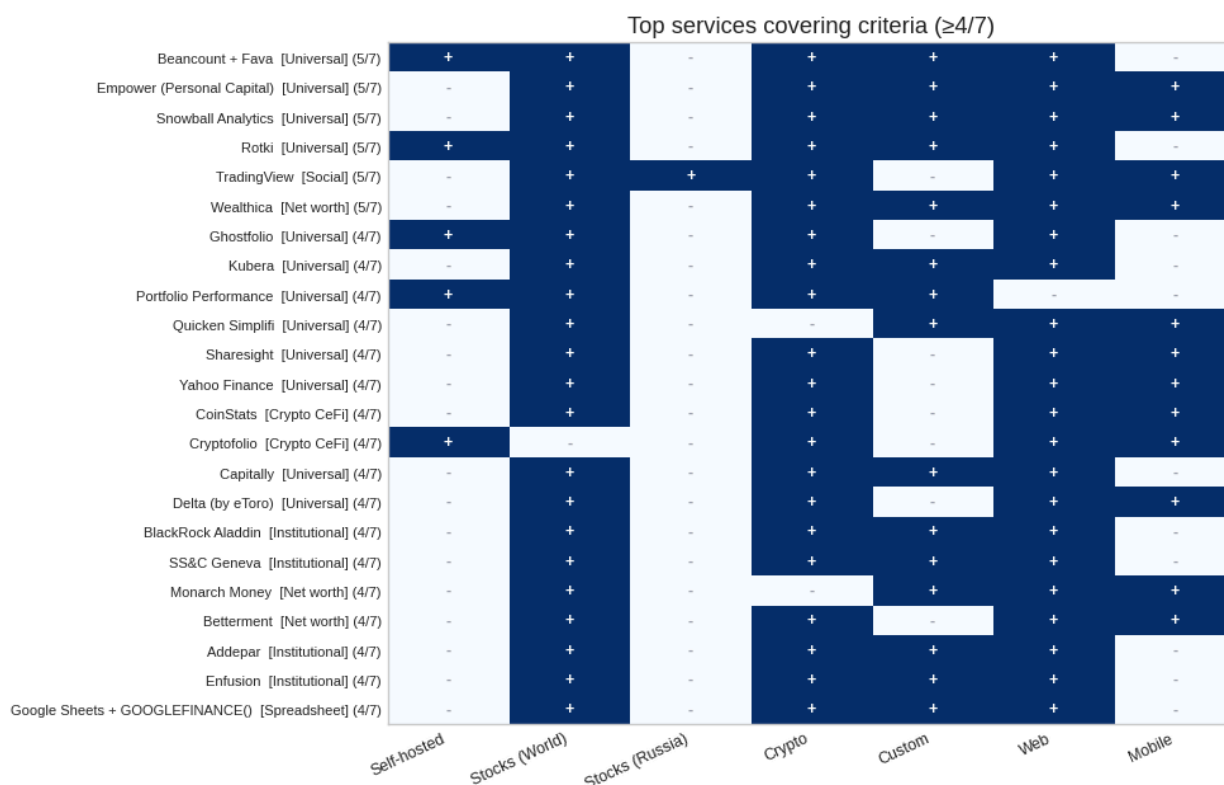
2.4 Analysis

We want to consider the services that best meet our requirements. For each of the seven service categories, we'll calculate the proportion of those that support each key feature. This will provide a general understanding of how fragmented the market is and where the gaps lie.



No single category provides us with a high share across all features simultaneously. The best one here is the Universal Tracker, which is logical. However, it has two major drawbacks: a low share in Russian stocks and self-hosted implementations.

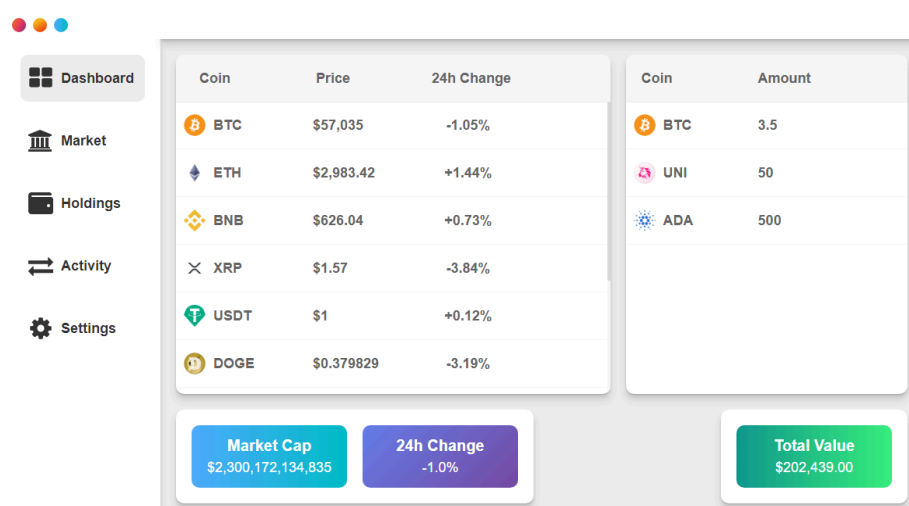
Let's simply try to find the service that will solve most of our problems. To do this, we'll extract all those that meet 4/7 criteria.



We obviously receive the smallest share of Russian Stocks. But this can be fixed if the solution can be further developed! So let's analyze top-ranked services that offer self-hosting and support for mobile devices and the web (and if we were to extend it to new platforms, we'd essentially be rebuilding the service). We have one such service - Cryptofolio [\[2\]](#).

Cryptofolio [\[2\]](#)

This solution has extremely limited internal functionality. This can be seen from its design, presented below. While it meets all the stated criteria, overall usability is completely lacking.



Conclusion

Thus, we come to the confirmation of our assumptions. Services that suit our needs don't exist, although there is demand for them. This is confirmed by the fact that the aforementioned Cryptofolio, despite its limitations, has 356 stars on its GitHub page.

3. How we implement the project

Everything described below should be deployed via Docker compose for quick configuration on the backend server and web application.

3.1 Frontend

The frontend needs to be cross-platform, so React Native is the obvious choice. To render price charts, we'll use `lightweight-charts` for the web version and `react-native-wagami-charts` for the mobile app. The state manager should be written in Zustand, as it's fairly lightweight, and large, complex states aren't expected on the frontend.

We also plan to implement personalized analytics charts that can be generated based on your positions and transactions. For this, we're using the `victory-native` library.

3.2 Backend

The basis will be Fastapi, which is preferable for essentially any project due to the combination of code simplicity and speed of operation.

We'll use Postgres to store information about transactions, positions, and so on. Migrations, which are essential because the code needs to be easily modifiable, will be performed using Alerich.

Celery and Redis are also required in this project for background price updates, portfolio snapshots, and other possible recurring tasks.

Prices are captured via pluggable APIs, which include at least the Moscow Exchange API, Coingecko API, and Yahoo API.

3.3 General structure

The application architecture is built around several key entities interconnected through transactions and price data.

Transactions are the fundamental unit of the system. Each transaction is recorded in the database and falls into one of three types. An input transaction is the entry of funds into the system: the user registers the receipt of an asset (for example, a deposit into a brokerage account or the receipt of cryptocurrency). A transfer transaction is the exchange of one asset for another, that is, the actual purchase or conversion. Transfer transactions are what create positions. An output transaction is the withdrawal of funds from the system: recording the sale or transfer of an asset outside the tracking loop.

Positions are created automatically based on transfer transactions. A position represents a record of ownership of a specific asset and stores information about the quantity, entry price, and the currency from which the transfer was made. For each position, the system tracks the change in the asset price and its quantity relative to the base currency since the position was created. This allows for the calculation of PnL, profitability, and other metrics at the individual position level.

Portfolios are user-defined groupings of positions. Users can combine positions into portfolios in any way, for example, by separating strategies or asset classes. Advanced analysis based on evidence-based finance is available for each portfolio, including calculation of risk metrics, correlations,

historical returns, and other indicators. Furthermore, the system aggregates all related transactions and portfolio assets to create portfolio dashboards that visualize analytics in a personalized manner.

Assets and Pricing APIs. Each asset in the system is linked to a pricing data source. Three built-in providers are supported: Moscow Exchange API (for Moscow Exchange stocks and bonds), Coingecko API (for cryptocurrencies), and YahooAPI (for international instruments). Connection to a provider is accomplished via an API key, if required by a specific service. In addition to the built-in providers, users can create a custom API - a custom pricing source implemented using the provided integration model. A custom API allows dynamic pricing updates for custom assets not included in the standard catalogs. A manual pricing mode is also available: prices for custom assets can be set manually without connecting to any API, which is useful for accounting for illiquid or non-marketable assets (such as real estate or business shares).

4. Selected analytical framework

Within the development of the portfolio management system, the primary analytical framework selected is evidence-based finance, which involves the use of strictly formalized and empirically confirmed models. This section is developed in collaboration with the financial minor team.

The core method is Modern Portfolio Theory (MPT) [3]. This theory represents a mathematical model of portfolio formation, where optimization is performed with respect to expected return and risk. The fundamental principle of MPT is that assets should not be considered in isolation, but rather in terms of their contribution to overall portfolio risk. Diversification reduces total risk due to imperfect correlation between asset returns.

The expected portfolio return is defined as:

$$E(R_p) = \sum_{i=1}^N w_i E(R_i)$$

where:

- w_i - weight of asset i in the portfolio
- $E(R_i)$ - expected return of asset [4]

The key result of the model is the construction of the efficient frontier - a set of portfolios that provide the maximum expected return for a given level of risk or the minimum risk for a given level of return. Thus, the optimization problem reduces to selecting the optimal allocation of asset weights.

Additionally, correlation analysis is used, which makes it possible to identify assets with low interdependence, ensuring real diversification. This is fundamentally important, since it is the covariance structure of assets that determines the overall portfolio risk, rather than only the individual characteristics of the instruments. The formula is:

$$\sigma_p^2 = \sum_{i=1}^N \sum_{j=1}^N w_i w_j \sigma_{ij}$$

where:

- σ_{ij} - covariance of returns of assets i и j [4]

To evaluate the fundamental value of assets, classical financial analysis models are integrated into the system, such as:

Discounted Cash Flow (DCF) - valuation based on discounting future cash flows. The formula is:

$$V_0 = \sum_{t=1}^T \frac{CF_t}{(1+r)^t}$$

where:

- CF_t - cash flow at time t
- r - discount rate

Gordon Model (Dividend Discount Model) — valuation of stocks through dividends. The formula is:

$$P_0 = \frac{D_1}{r-g}$$

where:

- D_1 - expected dividends in the next period
- r - required rate of return
- g - dividend growth rate

These models allow one to move away from market noise and speculative factors, forming an estimate of intrinsic value.

The Black–Scholes model is also used, which is an industry standard for option pricing and hedging instruments. It makes it possible to formalize the value of risk based on the stochastic behavior of asset prices [\[4\]](#). The formula is:

$$C = S_0 N(d_1) - X e^{-rt} N(d_2)$$

where:

- S_0 - asset price
- X - strike price
- $N(d)$ - normal distribution function

Thus, the selected analytical stack is characterized by the following advantages: reliance on validated academic models, interpretability of results, reproducibility of computations, and applicability to multi-asset portfolios. Unlike most existing solutions focused on trading and forecasting, the proposed approach is aimed at structural portfolio optimization and risk management, which makes it more robust and scientifically grounded.

5. Project success metrics

The evaluation of the effectiveness of the developed system is based on metrics that reflect both the distribution of the open-source solution and the activity of the community, as well as the quality of

development. Unlike classical SaaS products, within this project the metrics must be directly available through GitHub and internal system data, without the use of external analytics.

5.1. Popularity and distribution metrics

As basic indicators of interest in the project, GitHub metrics are used, characterizing the level of attention and distribution of the repository.

The key metric is the number of stars, reflecting the number of users who have marked the repository as interesting or useful. This metric is used as an indicator of project popularity and visibility [3]. At the same time, it is important to take into account that stars reflect interest rather than actual usage of the project. Additionally, the forks metric is used, which reflects the number of copies of the repository. Unlike stars, forks are a stronger signal of engagement, since they usually indicate the intention to use, modify, or develop the project. Thus, forks act as a proxy metric for the actual usage of the system.

5.2 Community activity metrics

For an open-source project, the presence of an active developer community is a critically important factor [6].

One of the key metrics is the number of contributors, that is, the number of unique users who have made changes to the codebase. This indicator reflects the openness of the project and its ability to attract external developers. Additionally, commit frequency is used, characterizing development activity. Regular changes in the codebase indicate that the project is maintained and evolving. An important metric is also the pull request rate, reflecting the number of proposed changes from the community. The presence and growth of pull requests indicate user engagement and their willingness to participate in the development of the system.

5.3 Final success criteria

The success of the project is determined based on achieving specific quantitative indicators characterizing the growth of popularity, usage, and community involvement. Within the project, the following benchmarks are proposed.

The project is considered to have reached a basic level of success under the following conditions:

- number of stars $\geq 100-300$, which corresponds to the stage of initial recognition of the project and going beyond a narrow circle of developers;
- number of forks $\geq 10-30$, which indicates the presence of users willing to use or modify the solution, since forks reflect deeper engagement than stars [7];
- ratio

$$\frac{Forks}{Stars} \geq 0.05$$

which indicates the transition from passive interest to practical usage (typical values for actively used projects range from 5-20%);

- number of contributors $\geq 3-5$, which means that the project ceases to be fully individual and begins to develop as an open-source initiative with external participation;
- presence of at least 1-2 pull requests per month, which indicates regular community activity and project development;
- proportion of accepted pull requests:

$$MergeRate = \frac{N_{merged\ PR}}{N_{PR}} \geq 0.6$$

which reflects the effectiveness of interaction with contributors and the absence of “blocking” in project development.

Thus, the success of the project is determined not by the absolute values of individual metrics, but by their joint dynamics and the achievement of threshold values reflecting the formation of a sustainable open-source solution.

6. Project objectives and expected results

The implementation of the project involves solving a set of interconnected tasks covering the full lifecycle of software product development - from the stage of architectural design to its publication and further development in an open-source environment [8].

At the first stage, the architectural foundation of the system is formed, oriented towards a self-hosted deployment model. This implies the development of a server-side component with API support, the organization of data storage, and the implementation of authentication and access management mechanisms. A key feature of this approach is the transfer of computational infrastructure to the user side, which ensures a high level of data control and independence from external services. At the same time, the possibility of reproducible system deployment is implemented, which is a necessary condition for open-source solutions.

The next stage is associated with the development of the user interface, providing access to the system functionality. Within this task, an application is formed that allows the user to manage an investment portfolio, track the state of assets, and interact with analytical tools.

Next is an analytical module implementing evidence-based finance methods. This module includes portfolio optimization algorithms, analysis of relationships between assets, and their valuation based on fundamental models. The functioning of the analytical layer is impossible without the proper organization of the process of data collection and processing. In this regard, a separate task is the integration with external sources of financial information, processing of time series, and ensuring data “freshness”.

The expected result of the project is the creation of a fully functional portfolio management system combining a self-hosted architecture, integration of various asset classes, and analytical tools based on scientifically grounded models. It is assumed that the developed solution will provide the user with a high level of control over data, improve the quality of investment decisions, and create a foundation for further development within the open-source ecosystem.

7. References

- [1] LLM-Generated Dataset for Financial Portfolio Analysis, 2026. [Google Drive](#)
- [2] Xtrendence. Cryptofolio: Open-source self-hosted cryptocurrency portfolio manager. [Github](#)
- [3] H. Markowitz, *Portfolio Selection*, Journal of Finance, 1952.

- [4] “Modern Portfolio Theory (MPT) implementation and formulas,” GitHub repository, 2026. [GitHub](#)
- [5] H. Borges et al., “On the Popularity of GitHub Applications,” arXiv, 2015. [arXiv](#)
- [6] Open Source Guides, “Starting an Open Source Project,” GitHub, 2026.
- [7] GitHub Documentation, “About forks,” 2026.
- [8] General software engineering resource, “What is SDLC,” GitHub Resources, 2026.